



Department: Information Technology

**Bachelor of Information and
Communications Technology (BINCTI) & Advanced
Diploma in Information and Communications
Technology (ADICTI)**

MODULE / SUBJECT CODE: PBDV30I | PBDE40I

Project 2 – Native Mobile Application

Due Date – Refer to the study guide.

General Information and Project Task

Collaborative projects afford students the opportunity to fully engage in their learning by communicating their needs, ideas, and opinions. This not only helps to cultivate competencies in various areas of software and application development but also prepares them for future employment opportunities and other projects.

Establishment of the Project Team

This is a group project evaluation. The project teams, consisting of 10–13 members per group, were established by the lecturer based on the responses gathered from the survey administered to students. The survey results were used to ensure balanced representation of diversity and inclusiveness in terms of backgrounds, perspectives, and capabilities within each group.

Each group is required to provide the following information:

1. **Group Name:** State a unique name that best represents your group.
2. **Group Leader:** Indicate the elected group leader who will coordinate team activities and communication.

Each group member must:

- Contribute to overall project objectives.
- Complete individual deliverables.
- Provide expertise.
- Work respectfully with other group members.
- Find solutions to problems and not make excuses.

Note that a member of one group may not be allowed to migrate to another group. Any unforeseen challenges should be discussed with the lecturer. While this project emphasizes group work, it is the responsibility of each individual student to ensure that they understand the curriculum for this module and the requirements of the project.

Project Themes

This section presents the selected project topics, the challenges they aim to address, and the proposed **mobile application solutions** for implementation. Each project is designed to tackle real-world problems affecting students, communities, and broader society, with a focus on usability, accessibility, and impact. Some projects aim to enhance everyday experiences such as learning, safety, and wellbeing, while others address wider issues such as employment, sustainability, and social inclusion. For each topic, a set of **core features** is outlined to define the key functionalities of the application, including essential system capabilities and AI-driven components that support intelligent and user-centred interactions.

The “Need for Solution” provided under each topic serves as a general guideline rather than a complete specification. It highlights the intended focus and key objectives of the project while allowing flexibility for further development. The listed **core features** provide a foundational structure for the system but are not exhaustive. Students are expected to critically analyse each problem, expand on the proposed direction, and enhance the application by adding innovative functionalities and improvements to develop a comprehensive, effective, and well-justified mobile application solution.

Group 6 & 32 - Smart Agriculture Assistant

Problem: Farmers lack real-time farming insights.

Need: Data-driven farming support.

Core Features: Weather updates, Crop management logs, Market price tracking, Alerts, Farm records, AI crop disease detection

Group 29 & 17 -Student Study Planner

Problem: Poor time management among students.

Need: Structured study planning tool.

Core Features: Timetable creation, Task management, Exam reminders, Progress tracking, Notes section, AI study schedule generator, AI performance insights

Group 24 & 3 -Smart Waste Management App

Problem: Inefficient waste collection and recycling.

Need: Better waste tracking and awareness.

Core Features: Waste reporting, Collection schedules, Recycling guides, Location mapping Notifications, AI waste image classification

Group 15 & 8 - Local Business Support Platform

Problem: Small businesses struggle with visibility.

Need: Platform to connect businesses with customers.

Core Features: Business listings, Product/service showcase, Reviews & ratings, Search/filter Promotions, AI recommendation system

Group 9 & 40 - Mental Health Companion

Problem: Limited access to emotional support.

Need: Private mental health assistance.

Core Features: Mood tracking, Journaling, Meditation tools, Alerts, Resource hub, AI emotional support chatbot, AI mood prediction

Group 33 & 12 - Smart Public Transport Tracker

Problem: Unpredictable transport schedules.

Need: Reliable commuting information.

Core Features: Route planning, Live tracking, Alerts, Fare calculator, Stop locator, AI delay prediction

Group 25 & 10 - Civic Engagement Platform

Problem: Low community participation.

Need: Easy way to engage citizens.

Core Features: Polls & surveys, Community discussions, Event listings, Notifications, Feedback system, AI sentiment analysis, AI moderation

Group 2 & 28 - Digital Health Assistant

Problem: Lack of accessible health guidance.

Need: Mobile-based health support.

Core Features: Symptom checker, Health tips, Appointment booking, Medication reminders, Clinic locator, AI diagnosis support, AI chatbot

Group 36 & 20 - Food Donation & Waste Reduction App

Group 1 & 14 - Problem: Excess food from campuses and restaurants often goes to waste.

Need: A mobile app to connect donors with people or charities in need.

Core Features: Food donation postings, Request system, Location mapping, Scheduling pickups, Notifications, AI donor-recipient matching, Feedback and rating system

Group 27 & 19 - Campus Parking Finder

Problem: Students and staff struggle to find parking on campus.

Need: A mobile app that shows real-time parking availability.

Core Features: Map of parking spaces, Availability status, Reservation system, Alerts for full/available spots, Notifications, AI parking prediction, User feedback

Group 13 & 21 - Online Learning Assistant

Problem: Students struggle with understanding content.

Need: Personalized learning support.

Core Features: Course materials, Video lessons, Quizzes, Notes, Progress tracking, AI tutor chatbot, AI content summarizer

Group 23 & 41 - Emergency Response Coordination App

Problem: Delayed emergency response in communities.

Need: Faster coordination system.

Core Features: Emergency alerts, Location tracking, Contact services, Incident logging, Notifications, AI risk prediction

Group 31 & 18 - Food Waste Reduction App

Problem: Excess food waste in households/businesses.

Need: Better food management solutions.

Core Features: Food inventory tracking, Expiry alerts, Donation system, Recipes, Notifications, AI food usage suggestions, AI waste prediction

Group 13 & 35 - Freelancer Marketplace App

Problem: Freelancers struggle to find clients.

Need: Platform for gig opportunities.

Core Features: Profile & portfolio, Job postings, Bidding system, Messaging, Reviews, AI job matching, AI pricing suggestions

Group 37 & 22 - Language Learning App

Problem: Difficulty learning new languages.

Need: Interactive and adaptive learning tools.

Core Features: Lessons, Vocabulary training, Speech practice, Quizzes, Progress tracking, AI speech recognition, AI personalized lessons

Group 11 & 39- University Resource Finder

Problem: Students struggle to find campus resources.

Need: Centralized campus information system.

Core Features: Facility directory, Map navigation, Event listings, Booking system, Notifications, AI search assistant, AI recommendation

Group 16 & 38 - Community Volunteer Platform

Problem: Students and community members struggle to find volunteer opportunities.

Need: A mobile app to connect volunteers with local causes.

Core Features: Volunteer opportunity listings, Profile and skills matching, Event registration, Feedback and ratings, Notifications, AI opportunity matching, AI volunteer impact tracking Chat and collaboration feature

Group 34 & 26 - Cognitive Support & Memory Aid App

Problem: Students with cognitive disabilities face challenges with memory, focus, and organization.

Need: A mobile app to provide reminders, cognitive exercises, and task support.

Core Features: Task and schedule reminders, Interactive brain exercises, Daily routine planner, Notifications, Progress tracking, AI personalized reminders, AI cognitive performance suggestions, Alerts for missed tasks

Group 30 & 20 - Community Skill Exchange Solution

Problem: People have skills but no platform to exchange them locally.

Need: A system that connects people for skill-sharing without money.

Core Features: User skill profiles, Skill request/post system, Matchmaking between users, Rating & feedback, Chat system, AI skill matching engine, AI trust/reputation scoring

Group 4 & 7 Sustainable Campus Projects Tracker

Problem: Campus sustainability projects are poorly tracked and managed.

Need: Centralized monitoring of green initiatives.

Core Features: Project listing & details, Progress tracking, Team collaboration, Document uploads, Notifications, AI progress analysis, AI sustainability impact prediction

Group 5 & 42 Campus Security Alert System

Problem: Slow response to emergencies on campus.

Need: Real-time alert and safety system.

Core Features: Panic/emergency button, Live location sharing, Alert broadcasting, Incident reporting, Security contact directory, AI threat detection patterns, AI alert prioritization

Project Objective

The objective of this project is to design and develop a mobile application that addresses the assigned challenge. Each group is required to analyse the allocated problem, explore feasible mobile solution approaches, and implement a functional, user-centred application that effectively meets the identified needs. The project should focus on mobile usability, responsiveness, and accessibility, ensuring the application can be deployed on Android and/or iOS platforms.

Integration of Emerging Technologies

In an era dominated by rapid technological evolution, incorporating emerging technologies such as Machine Learning (ML), Artificial Intelligence (AI) Assistants, Natural Language Processing (NLP), Cloud Computing and Data Analytics, can significantly enhance the functionality and market relevance of an application. Each group is required to explore and integrate these advanced technologies within their application.

Methodology

- The project must follow the Scrum Agile Development Framework, promoting iterative development, continuous feedback, adaptability, and effective team collaboration.
- Students are expected to create product backlogs, sprints, daily stand-ups, sprint reviews, and retrospectives.
- Each sprint should produce a working increment of the mobile app, progressively adding features and functionality based on feedback.

Technology Stack

Core Development Technologies

- React Native, Flutter, or Kotlin/Java/Swift (depending on platform) for responsive, cross-platform mobile interfaces.
- Mobile-friendly design principles; frameworks such as Material UI or Flutter Widgets for consistency.

- **Programming Languages:** Dart (for Flutter), Java/Kotlin (for Android), Swift (for iOS), or JavaScript/TypeScript (for React Native).
- **Client-side validation** to improve user experience and reduce invalid inputs.

Database

- Relational (MySQL, PostgreSQL, SQLite) or NoSQL (Firebase, MongoDB) depending on application requirements.
- Proper database design principles, including normalization, relationships, and scalability considerations, must be applied.
- RESTful APIs to facilitate structured communication between the mobile frontend and backend; proper HTTP methods (GET, POST, PUT, DELETE) and status codes should be implemented.

External API Integration

- Optional integration of third-party services to enhance functionality, such as:
 - Email or push notifications (e.g., Firebase Cloud Messaging)
 - SMS gateways
 - OAuth authentication (Google, Facebook, or institutional login)
 - Mapping and location-based services (Google Maps, Mapbox)
 - Payment gateways (if required)

Authentication and Authorisation

- Implement secure user authentication and role-based access control (e.g., admin, user).
- Ensure session management and token-based security (e.g., JWT).

Data Management and Security

- Applications collecting personal information (e.g., names, contacts, health data, complaints) must comply with POPIA principles.
- Secure password storage using hashing
- Input validation and sanitisation to prevent common mobile vulnerabilities.
- Encrypted local storage where sensitive data is stored on the device.

Development and Deployment Tools

- Source Control: Git for version management and collaboration.
- Testing: Basic unit testing, integration testing, and device testing.
- Deployment: The app will run locally on phones or emulators, and deployment to the app store is not required.

System Design Expectations

- Modular code structure following best mobile development practices.
- Clear file organisation for maintainability.
- Integration of AI components where required (e.g., image recognition, predictive analysis, chatbots).

Groups are encouraged to apply **analytical thinking, technical expertise, and creativity** to produce mobile applications that make a meaningful impact on communities and broader societal challenges. Students should leverage **Scrum practices** to ensure iterative development, continuous improvement, and high-quality delivery of mobile solutions.

Documentation Requirements

Proper documentation is essential to guide the development, testing, deployment, and maintenance of mobile applications. It ensures clarity, consistency, and reproducibility, and demonstrates a structured approach for evaluating the project. This guide outlines the key documentation components required for the mobile app projects, including design, technical details, AI integration, and user instructions.

Table of Contents

- A clear, organized list of all sections and subsections with page numbers.
- Include figures, tables, and appendices.

1. Introduction (1 page)

- Project objectives: Define the problem, purpose, and goals of the mobile application.
- Target audience: Identify who will use the app (e.g., students, community members, people with disabilities).
- High-level overview: Briefly describe the app's functionality, workflow, and impact.
- AI Component Overview: Describe the role of AI in the application (e.g., prediction, image recognition, translation).
- Scrum Implementation: Outline how Scrum methodology is applied, including sprint structure and team roles.

2. Materials and Methods (2–3 pages)

2.1 Architecture Overview

- Diagrams of the mobile app architecture, including frontend, backend, database, and AI components.
- Description of data flow, API interactions, and integration points.
- Include modular design details and how separation of concerns is achieved.

2.2 Technology Stack

- Mobile platforms supported (Android and/or iOS).
- Programming languages, frameworks (React Native, Flutter, Java/Kotlin/Swift).
- Libraries, UI components, AI tools (TensorFlow Lite, PyTorch Mobile, OpenAI APIs).
- Database selection (SQLite, Firebase, MySQL, PostgreSQL) and rationale.

2.3 APIs and External Services

- List and describe all APIs used.
- Include request/response formats, authentication, endpoints, and any third-party services (e.g., mapping, notifications, OCR, translation).

3. User Guide (2 pages)

- Screenshots of all screens, highlighting UI/UX flow.
- Step-by-step workflow showing how a user interacts with the app.
- Instructions for AI-enabled features (e.g., how to capture images for disease detection, use translation feature).
- Device/emulator instructions for running the app locally.

4. Source Code Overview (2 pages)

- Explanation of the folder structure and organization.
- Key classes, methods, and modules with annotations explaining functionality.
- Highlight critical code snippets, including AI model integration, API calls, and major logic functions.
- Include comments on maintainability and modularity.

5. Testing and Quality Assurance (2–3 pages)

- Testing strategies: Unit testing, integration testing, system testing, and device/emulator testing.
- Test cases: Include inputs, expected outputs, and actual results.
- Bug reports and fixes.
- Performance testing: App responsiveness, load time, and memory usage.
- Security considerations: Authentication, data privacy, and POPIA compliance.

6. Scrum Artifacts and Project Management (1–2 pages)

- Include product backlog, sprint backlog, sprint review outcomes, and retrospectives.
- Brief description of team roles and responsibilities.
- Evidence of iterative development and feature delivery.

7. References

- Cite all technologies, APIs, frameworks, and external resources used.
- Use a consistent citation style.